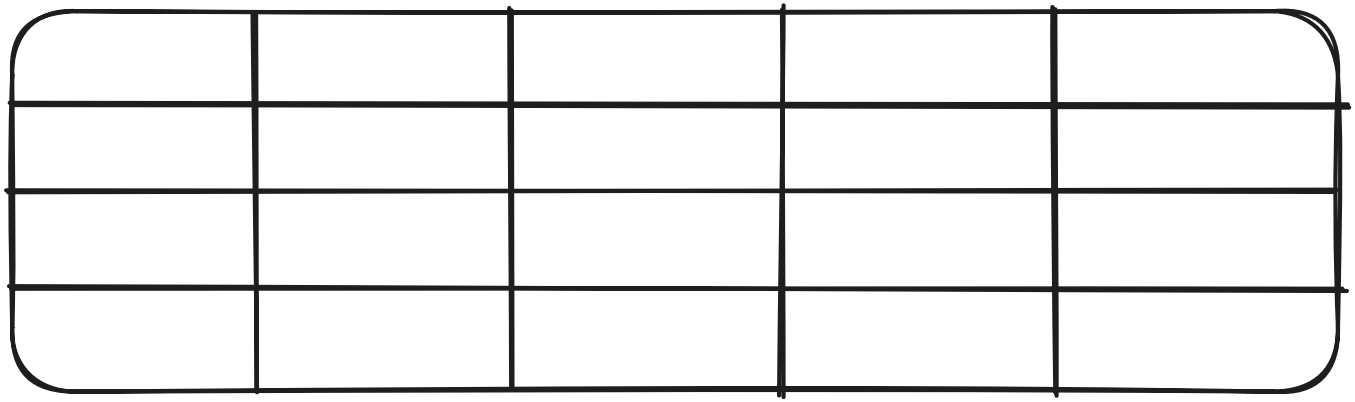
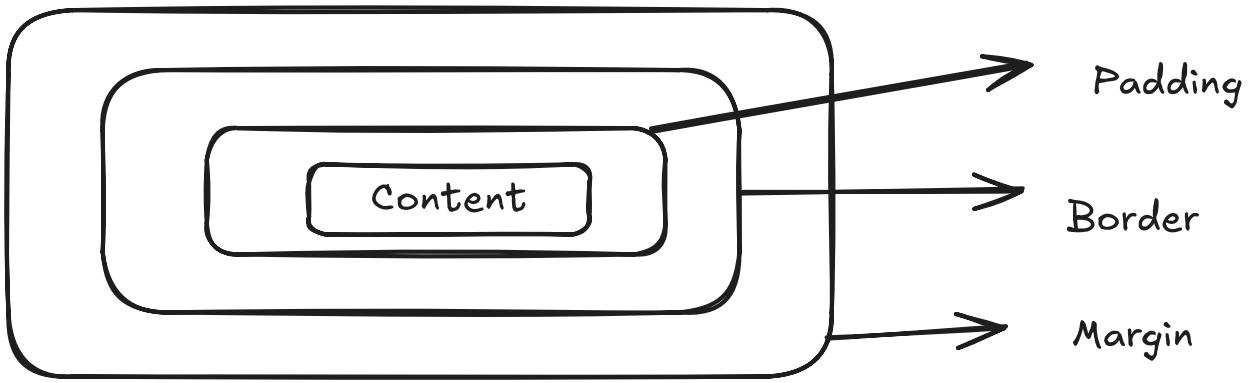
Topic: CSS Flexbox and Layout
Mentor: Sushant Goyal
Date: 30 August 2025

Topics:
1. CSS Flex-box
2. Layouts
3. Media Query



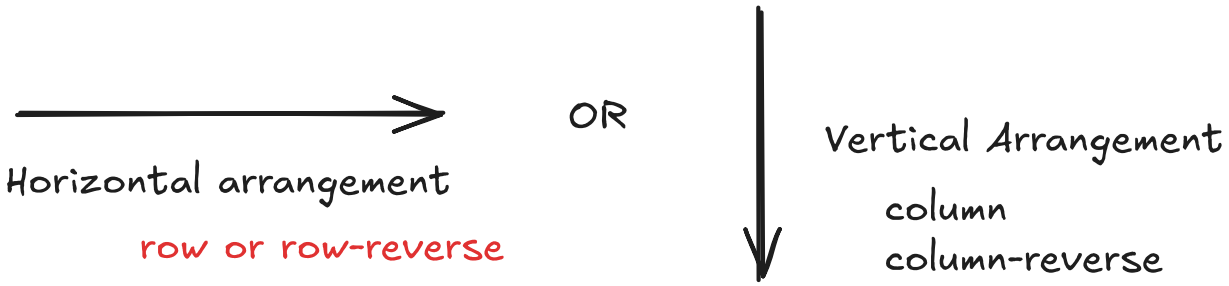
CSS:
1. Padding - to add the space around the content within the box
2. Margin - to add the space around the box.



Box Model

3. CSS Positions -
    a. static - by default static is applied. it keeps the flow of content intact.
    b. fixed - once applied, it will remain in same position and the element is brought out the flow of HTML
    c. relative - it is similar to static but we can apply top, left, right or bottom properties and the space will be reserved for the element.
    d. absolute - it takes element out of the flow and position it in relative or absolute position of the nearest parent.
4. Colors:
    a. Solid colors - adding color by name
    b. HEX - using #<COLOR_CODE>, can be changed or check from the inspect
    c. HSL - using hue, saturation and lightness. hsl(hue, saturation, lightness)
    d. RGB - red, green and blue color. rgb(red_value, green_value, blue_value)
    e. RGBA - red, green, blue along with alpha (transparency)
        rgba(red, green, blue, transparency/alpha)

CSS Flex Box
it is a layout model that arrange elements in single dimension (1-D)



It helps in distributing the available space on the viewport or defined by the parent
to easily align even if the size is not known or is varying.

using flex property it will keep the child elements in the confined area it has access to.

to use flexbox, display: flex is used.

inline-flex: it occupies the area which is required by the content in contrast to flex which occupies the complete width of the parent element or the viewport.

Whenever display:flex is applied to an element, its child elements become flex-items and
flex-items are the direct children of the element.
it will not be applied grand-child elements.

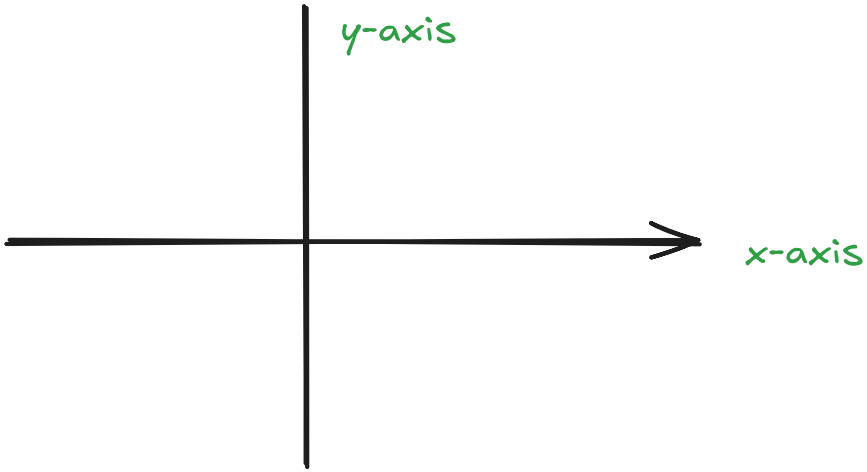
```
<div class="container">
        <div class="box">
            <p class="paragraph">Some text</p>
        </div>
        <div class="box">2</div>
        <div class="box">3</div>
    </div>
```

Flex items will not be applicable to paragraph

using flex-direction we can control the main axis direction

```
{
    display: flex;
    flex-direction: row // default value
}
```

flex-direction: row-reverse - to rotate around the y-axis
column - to show items vertically
column-reverse - to rotate the items around x-axis



flex-wrap: determines whether items stay in one line or wrap.
    default is set to nowrap,
    flex-wrap: wrap: to wrap the contents.

justify-content: it was aligning items across main axis
1. start - items packed to the start
2. flex-start - similar to start
3. end - items packed to the end
4. flex-end - similar to end
5. left - it will move items to the left side
6. right - will move items to the right side
7. space-between - items evenly spread, no space on the ends
8. space-around - equal spaces around all items (ends gets halved)
9. space-evenly - equal space between and around all items

align-items: it is aligning items across cross axis.
1. end: it align items to end of cross axis.
2. flex-end: similar to end
3. start or flex-start - keeps the items to start of cross axis
4. center - keeps the items in the center of cross
5. stretch - fill the container height or width
6. baseline - align items on text baseline

align-content: control alignment of multiple rows/columns. it works only if we have
                multi-line flex containers

** if we have flex-wrap as nowrap, we can not use align-content
1. flex-start - all rows packed to start or top
2. flex-end - all rows packed to end or bottom
3. center - center the rows or the content
4. space-between - even space around the rows
5. space-evenly - equal space between and around the rows
6. space-around - even space around the rows

Flex Items Properties:
1. flex-grow: defines how much a flex item should grow relative to others.
flex-grow: <number>;
example: an item with flex-grow:2 grows twice as much as one with flex-grow:1

2. flex-shrink: defines how much an item should shrink if needed
default value is 1. // flex-shrink:1
using flex-shrink:0 prevents shrinking

3. align-self: it will override the align-items property defined by the flex for individual flex item.

```
{
    align-items: <property>
}
```

4. order: changes the order of flex items visually (DOM is not altered)
higher the order the later it will appear in the flex container

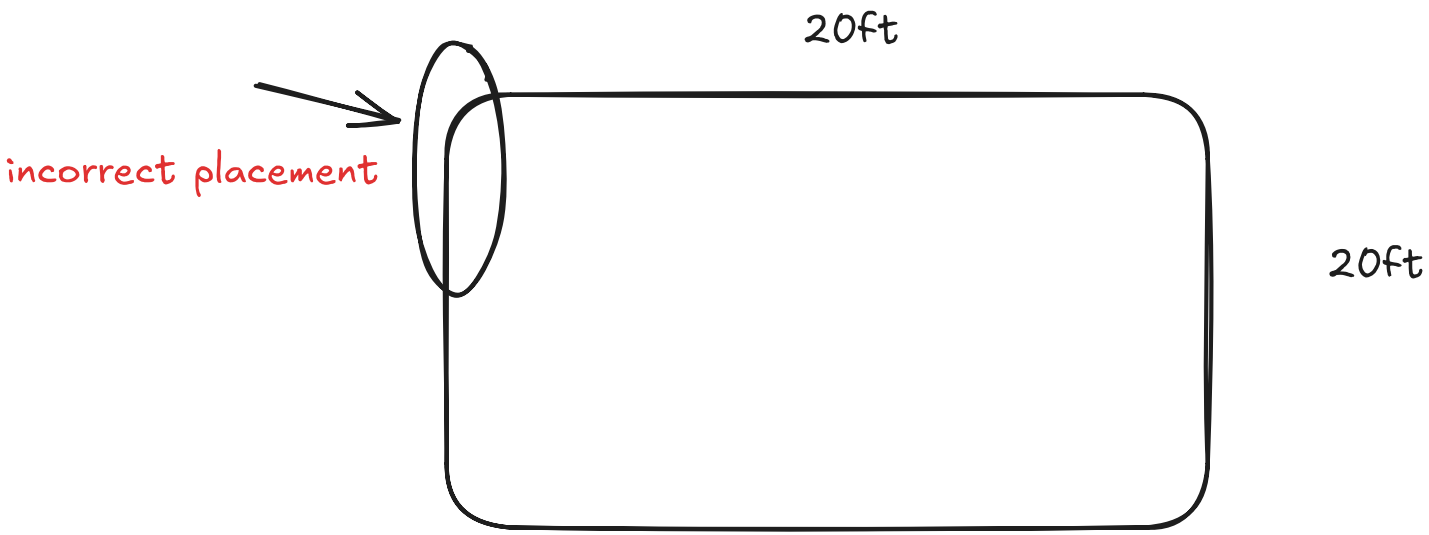
5. flex: it is a shorthand of flex-grow and flex-shrink and flex-basis

flex: 1 // flex-grow: 1 and flex-shrink: 1 and flex-basis: 0
flex: <flex_grow> <flex_shrink> <flex_basis>

6. flex-basis: it defined the initial size of the flex item before growing or shrinking
   flex-basis: 200px;

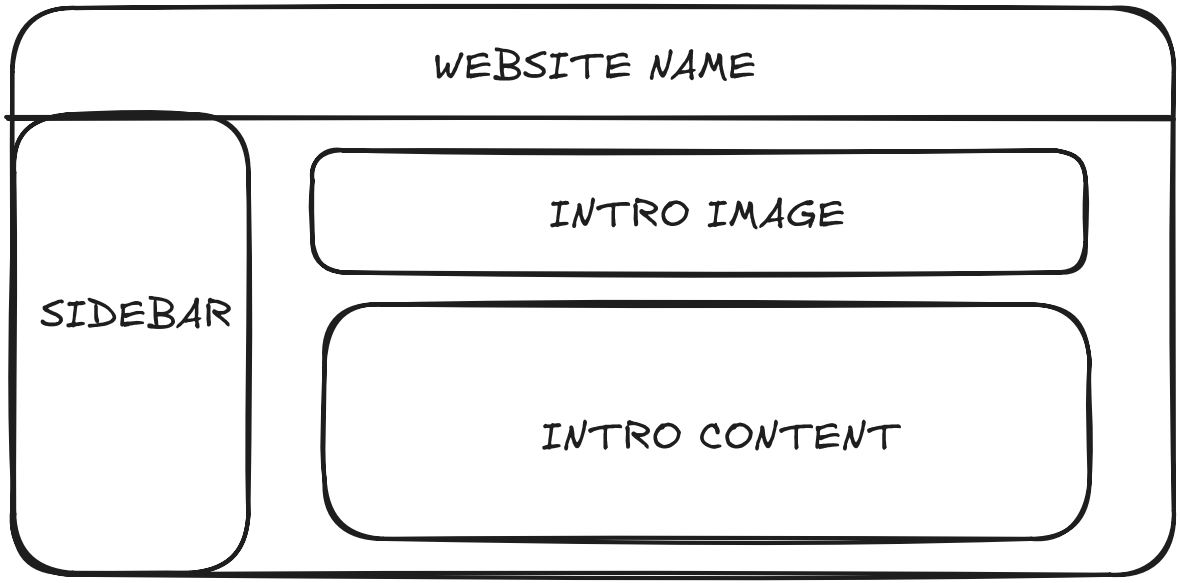
Layout Designing:

using flex-box and css

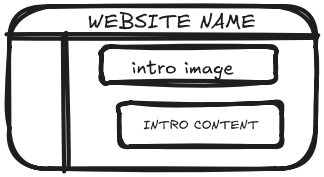


We moved from a desktop area to a responsive area
1. Desktop layout
2. Tablet layout
3. Mobile Layout

Desktop to mobile pages



Desktop Layout



Mobile Layout

Media Queries

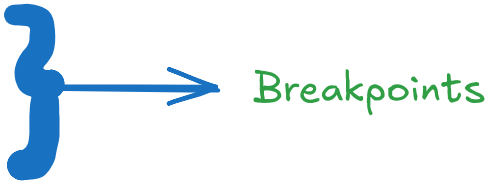
it let us define the viewport sizes or the device width and add style accordingly.

it lets us define the CSS styles conditionally based on the characteristics of device

```
@media media-type (condition) {
    // CSS rules
}
```

@media - starts the media query
media-type - defines the type of the condition - screen, all, print
condition - max-width or max-height

max-width: 768px -> tablet layout
max-width: 480px -> mobile layout
max-width: 1024px -> small screens



1. Flexbox and how it is used
2. How to create a layout using flexbox
3. media queries.

Task:
1. layout the portfolio in 2 columns

1. layout the portfolio in 2 columns
2. add a sticky navigation bar
3. add footer section with social profiles - linkedin and github